Mini-projet d'algorithmique avancée

Triangulation minimale d'un polygone convexe

ENSSAT INFO 2 – 2025-2026

Implémentation en Rust

Table des matières

1 Introduction **2**

2 A. Questions préliminaires **2**
2.1 A.1 : Nombre de cordes distinctes dans un polygone à n sommets 2
2.2 A.2 : Toutes les triangulations comportent le même nombre de cordes . . . 2

3 B. Essais successifs **3**
3.1 B.1 : Fonction `validecorde(i, j)` 3
3.2 B.2 : Analyse de la stratégie naïve par sommets 4
3.3 B.3 : Stratégie correcte, itération sur les cordes 5

4 C. Programmation dynamique **7**
4.1 C.1 : Formule de récurrence 7
4.2 C.2 : Algorithme 7
4.3 C.3 : Complexité 8
4.4 C.4 : Intérêt du sommet commun 8

5 D. Algorithme glouton **9**
5.1 D.1 : Mise en œuvre 9
5.2 D.2 : Caractère exact 9

6 E. Recommandation argumentée **11**

7 Conclusion **12**

Annexe — Architecture du code **12**

Introduction

Ce projet étudie le problème de la **triangulation minimale d'un polygone convexe**. Étant donné un polygone plan à n sommets $s_0, s_1, \ldots, s_{n-1}$ (classés en ordre rétrograde), il s'agit de sélectionner un ensemble de **cordes** (segments joignant deux sommets non adjacents) tels que :

1. les cordes ne se croisent pas,
2. le polygone est entièrement subdivisé en triangles,
3. la somme des longueurs des cordes choisies est **minimale**.

Nous étudions trois approches algorithmiques : les essais successifs (backtracking), la programmation dynamique et l'algorithme glouton. Le code est implémenté en **Rust** pour ses garanties de performance et de correction mémoire.

Exemple de référence : l'heptagone de l'énoncé (sommets donnés en annexe). La triangulation non minimale proposée dans l'énoncé a un poids de 77,56 ; la triangulation optimale a un poids de **75,43** (cordes (s_0, s_2), (s_0, s_5), (s_2, s_5), (s_3, s_5)).

A. Questions préliminaires

A.1 : Nombre de cordes distinctes dans un polygone à n sommets

Le nombre total de paires de sommets est $\binom{n}{2} = \frac{n(n-1)}{2}$. Parmi ces paires, n sont des **arêtes** du polygone (sommets adjacents). Le nombre de cordes (paires non adjacentes) est donc :

$$\text{nombre de cordes} = \binom{n}{2} - n = \frac{n(n-1)}{2} - n = \frac{n(n-3)}{2}$$

Le tableau suivant récapitule, pour quelques valeurs de n, le nombre total de cordes $\left(\frac{n(n-3)}{2}\right)$ ainsi que le nombre de cordes dans toute triangulation ($n-3$, démontré en A.2) :

n	Cordes possibles $\left[\frac{n(n-3)}{2}\right]$	Cordes par triangulation $[n-3]$
4	2	1
5	5	2
6	9	3
7	14	4
10	35	7

A.2 : Toutes les triangulations comportent le même nombre de cordes

Théorème : Toute triangulation d'un polygone convexe à n sommets comporte exactement $n-3$ cordes.

Preuve par récurrence sur n.

— **Base ($n = 3$) :** Le triangle est déjà triangulé sans aucune corde. $0 = 3 - 3$.

— **Hérédité :** Supposons le résultat établi pour tout polygone convexe à $k < n$ sommets. Soit P un polygone convexe à n sommets et T une triangulation de P.

Considérons le triangle de T qui contient l'arête (s_0, s_1). Il a la forme (s_0, s_1, s_k) pour un certain $k \in \{2, \ldots, n-1\}$. Retirer s_1 produit un polygone convexe P' à $n-1$ sommets. La restriction de T à P' comporte exactement **une corde de moins** que T, selon le cas :

— **Cas $k = 2$:** La corde (s_0, s_2) de T devient une *arête* de P' (s_2 est désormais adjacent à s_0) ; elle n'est plus comptée comme corde dans $T \cap P'$.

— **Cas $k \geq 3$:** La corde (s_1, s_k) de T *disparaît* dans P' (le sommet s_1 est retiré), tandis que (s_0, s_k) demeure une corde de P'.

Dans les deux cas, $T \cap P'$ est une triangulation de P' comportant, par hypothèse de récurrence, $(n-1) - 3 = n - 4$ cordes. Donc T comporte $n - 4 + 1 = \mathbf{n - 3}$ cordes.

□

B. Essais successifs

B.1 : Fonction `validecorde(i, j)`

La fonction `valid_chord(i, j, drawn, vertices)` retourne `vrai` si et seulement si :

1. La corde (s_i, s_j) n'a pas déjà été tracée (vérification dans `drawn`).
2. La corde (s_i, s_j) ne coupe aucune corde déjà tracée (test d'intersection par produits vectoriels).
3. *(Pour généralité)* La corde ne coupe aucune arête du polygone, condition automatiquement vérifiée pour un polygone strictement convexe.

L'intersection de deux segments est détectée par la méthode des **produits vectoriels signés** : deux segments (A, B) et (C, D) se coupent proprement si et seulement si C et D sont de part et d'autre de la droite AB, et réciproquement A et B sont de part et d'autre de la droite CD.

```
Fonction validecorde(i, j, tracees, sommets) :
  Pour chaque corde (a, b) dans tracees :
    Si (a = i et b = j) ou (a = j et b = i) :
      retourner FAUX  // deja tracee

  Pour chaque corde (a, b) dans tracees :
    Si a != i et a != j et b != i et b != j :
      Si segments_croisent(sommets[i], sommets[j], sommets[a], sommets[b]) :
        retourner FAUX

  retourner VRAI
```

Pseudo-code de `validecorde`

B.2 : Analyse de la stratégie naïve par sommets

La stratégie naïve traite les sommets en ordre croissant : à l'étape i, elle trace une corde valide depuis s_i ou ne trace rien.

La stratégie naïve génère des doublons. La corde (s_i, s_j) peut être tracée à l'étape i (depuis s_i) ou à l'étape j (depuis s_j). Deux branches différentes de l'arbre de recherche peuvent donc produire la même triangulation via des ordres de tracé différents.

Exemple : Sur le pentagone, $\{(s_0, s_2), (s_2, s_4)\}$ peut être trouvée par (étape 0 : (s_0, s_2), étape 2 : (s_2, s_4)) ou par (étape 0 : rien, étape 2 : (s_0, s_2) vu depuis s_2, étape 4 : (s_2, s_4)).

La stratégie naïve est incomplète. Sous l'interprétation canonique où la corde (s_i, s_j) $(i < j)$ n'est disponible qu'à l'étape i, la stratégie ne peut tracer qu'une seule corde par étape. Or certaines triangulations requièrent plusieurs cordes issues du même sommet s_i comme plus petite extrémité.

Contre-exemple : Pour l'hexagone ($n = 6$), la triangulation en éventail depuis s_0 : $T = \{(s_0, s_2), (s_0, s_3), (s_0, s_4)\}$ comporte 3 cordes dont s_0 est la plus petite extrémité. Elles ne peuvent être tracées qu'à l'étape 0, mais la stratégie n'en trace qu'une seule. Cette triangulation est donc inaccessible.

B.3 : Stratégie correcte, itération sur les cordes

B.3.a : Algorithme

Pour garantir que chaque triangulation est construite **une fois et une seule**, on énumère les cordes dans un ordre fixé (triées par longueur croissante) et on décide pour chacune de l'inclure ou non. Cela forme un arbre binaire de décision sans ambiguïté.

```
Algorithme Backtrack(cordes, start, selection, meilleures, meilleurCout) :
  Si |selection| = n - 3 :
    // Elagage 1 (completude) : n-3 cordes non-croisees => triangulation complete
    cout <- somme des longueurs dans selection
    Si cout < meilleurCout :
      meilleurCout <- cout
      meilleures   <- selection
    Retourner

  restantes  <- |cordes| - start
  necessaires <- (n - 3) - |selection|

  // Elagage 2 : pas assez de cordes restantes pour completer
  Si restantes < necessaires : Retourner

  // Elagage 3 (branch-and-bound) : cout partiel deja trop eleve
  Si somme(selection) >= meilleurCout : Retourner

  Pour idx = start a |cordes| - 1 :
    // Elagage 4 (borne inferieure) : meme en prenant les k cordes les plus courtes
    //   restantes, la solution ne peut pas battre le meilleur connu
    borne <- somme(selection) + somme des k prochaines longueurs
    Si borne >= meilleurCout :
      break  // toutes les cordes suivantes sont encore plus longues => inutile

    Si validecorde(cordes[idx], selection, sommets) :
      selection.ajouter(cordes[idx])
      Backtrack(cordes, idx+1, selection, meilleures, meilleurCout)
      selection.retirer(cordes[idx])
```

Pseudo-code de l'algorithme backtracking

B.3.b : Complexité

Sans élagage, l'algorithme explore tous les sous-ensembles de taille $n-3$ parmi m cordes, soit $\binom{m}{n-3}$ appels. L'arbre binaire donne $O(2^m)$ appels en pire cas, avec $m = \frac{n(n-3)}{2}$. La complexité est donc :

$$O\left(2^{n(n-3)/2}\right)$$

B.3.c : Conditions d'élagage et gains

Quatre conditions d'élagage sont implémentées :

1. **Complétude :** Lorsque $n-3$ cordes non-croisées ont été sélectionnées, la triangulation est complète. On enregistre la solution si elle améliore le meilleur coût connu, puis on retourne.

2. **Insuffisance :** Si le nombre de cordes restantes dans la liste est inférieur au nombre de cordes encore nécessaires ($n - 3 - |\text{sélection}|$), il est impossible de compléter la triangulation.
3. **Branch-and-bound :** Si le coût partiel de la sélection courante est déjà supérieur ou égal au meilleur coût connu, toute extension ne peut qu'aggraver la situation.
4. **Borne inférieure :** Grâce au tri par longueur croissante, les k prochaines cordes sont les plus courtes disponibles. Si même en les incluant toutes le coût total dépasse le meilleur connu, on coupe la *boucle entière*, car les cordes suivantes ne peuvent être que plus longues.

La condition 4 est de loin la plus efficace : elle exploite l'ordre de tri pour arrêter la boucle dès que toute continuation est sous-optimale.

Tableau comparatif (polygone régulier) :

n	Appels (cond. 1-3)	Appels (cond. 1-4)	Gain
7	191	17	×11
10	20 791	207	×100
13	2 596 797	4 516	×575
20	$> 10^9$	10 338 656	—

B.3.d : Limite pratique

Avec les quatre conditions, le temps croît d'un facteur $\approx 3{,}2$ par incrément de n.

n	Appels	Temps (ms)	Temps (s)
10	207	0	0,000
13	4 516	1	0,001
15	38 014	9	0,009
17	274 976	80	0,080
18	927 798	302	0,302
19	3 173 616	1 032	1,032
20	10 338 656	3 405	3,405

Par extrapolation depuis $n = 20$ ($\approx$ 3 400 ms) avec le facteur 3,2 : $n = 22 \approx 33$ s, $n = 23 \approx 106$ s < 2 min, $n = 24 \approx 340$ s > 2 min. **La limite pratique est donc $n \approx 23$ en moins de 2 minutes.**

C. Programmation dynamique

C.1 : Formule de récurrence

On définit $T(i,t)$ comme le coût minimal en cordes de la triangulation du sous-polygone $s_i, s_{i+1}, \ldots, s_{i+t-1}$ (indices modulo n). Le segment de base (s_i, s_{i+t-1}) est exclu du coût car il est soit une arête du polygone, soit une corde déjà comptabilisée par le problème père.

Cas de base :

$$T(i,1) = T(i,2) = 0$$

Cas récursif ($t \geq 3$) : On choisit un sommet s_{i+k} ($k \in \{1, \ldots, t-2\}$) pour former le triangle $(s_i, s_{i+k}, s_{i+t-1})$. Les nouvelles cordes introduites sont :

- (s_i, s_{i+k}) si $k \geq 2$ (sinon c'est l'arête $s_i \to s_{i+1}$),
- (s_{i+k}, s_{i+t-1}) si $k \leq t-3$ (sinon c'est l'arête $s_{i+t-2} \to s_{i+t-1}$).

$$T(i,t) = \min_{k=1}^{t-2} \Big[\mathbf{1}_{k\geq 2} \cdot d(s_i, s_{i+k}) + \mathbf{1}_{k\leq t-3} \cdot d(s_{i+k}, s_{i+t-1}) + T(i,\, k+1) + T(i+k,\, t-k) \Big]$$

La réponse est $T(0,n)$.

C.2 : Algorithme

```
Algorithme DP_Triangulation(sommets[0..n-1]) :
  // Precalcul des distances
  Pour i, j = 0 a n-1 : d[i][j] <- distance(sommets[i], sommets[j])

  // Initialisation
  Pour i = 0 a n-1 : dp[i][1] <- 0  ;  dp[i][2] <- 0

  // Remplissage par taille de sous-probleme croissante
  Pour t = 3 a n :
    Pour i = 0 a n-1 :
      dp[i][t] <- +inf
      Pour k = 1 a t-2 :
        ik  <- (i + k)     mod n
        it1 <- (i + t - 1) mod n
        c1  <- d[i][ik]   si k >= 2,   sinon 0
        c2  <- d[ik][it1] si k <= t-3, sinon 0
        cout <- c1 + c2 + dp[i][k+1] + dp[ik][t-k]
        Si cout < dp[i][t] :
          dp[i][t]    <- cout
          choix[i][t] <- k

  Retourner dp[0][n]
```

Pseudo-code de la programmation dynamique

La reconstruction des cordes parcourt `choix` récursivement depuis $(0,n)$.

C.3 : Complexité

Spatiale : $O(n^2)$, pour les tableaux `dp`$[n][n+1]$ et `choix`$[n][n+1]$.

Temporelle :

$$\sum_{i=0}^{n-1}\sum_{t=3}^{n}(t-2) = n \cdot \sum_{j=1}^{n-2} j = n \cdot \frac{(n-2)(n-1)}{2} = O(n^3)$$

Complexité temporelle : $\mathbf{O(n^3)}$ comparaisons.

C.4 : Intérêt du sommet commun

Les sous-problèmes $T(i, k+1)$ et $T(i+k, t-k)$ partagent le sommet s_{i+k}. Cela garantit que leurs zones respectives sont disjointes et contiguës, formant ensemble une triangulation valide sans vérification d'intersection supplémentaire. De plus, chaque triangulation est examinée **exactement une fois** : il y a une bijection entre les arbres de décomposition et les triangulations.

Si la décomposition utilisait des cordes arbitraires, l'état d'un sous-problème ne pourrait plus être caractérisé par un simple couple (i, t) mais nécessiterait un sous-ensemble quelconque de sommets (2^n états possibles), rendant la mémoïsation exponentiellement plus coûteuse.

Résultats sur l'heptagone :

$$\text{DP : coût} = 75{,}4307, \quad \text{cordes} = \{(0,2),\ (0,5),\ (2,5),\ (3,5)\}, \quad \text{temps} \approx 6\,\mu\text{s}$$

D. Algorithme glouton

D.1 : Mise en œuvre

L'algorithme maintient un anneau `ring` de sommets restants (initialement $0, 1, \ldots, n-1$). À chaque étape :

1. Pour chaque sommet s_i de l'anneau, calculer la longueur de la corde extérieure (s_{i-1}, s_{i+1}).
2. Sélectionner l'oreille de longueur minimale.
3. Enregistrer la corde (s_{i-1}, s_{i+1}), retirer s_i de l'anneau.
4. Répéter jusqu'à ce qu'il reste 3 sommets.

```
Algorithme Glouton(sommets[0..n-1]) :
  ring <- [0, 1, ..., n-1]
  cordes <- []  ;  cout_total <- 0

  Tant que |ring| > 3 :
    meilleure_longueur <- +inf
    meilleure_oreille  <- -1

    Pour i = 0 a |ring| - 1 :
      prev <- ring[(i - 1) mod |ring|]
      next <- ring[(i + 1) mod |ring|]
      longueur <- d(sommets[prev], sommets[next])
      Si longueur < meilleure_longueur :
        meilleure_longueur <- longueur
        meilleure_oreille  <- i

    prev <- ring[(meilleure_oreille - 1) mod |ring|]
    next <- ring[(meilleure_oreille + 1) mod |ring|]
    cordes.ajouter((prev, next))
    cout_total <- cout_total + meilleure_longueur
    ring.supprimer(meilleure_oreille)

  Retourner (cout_total, cordes)
```

Pseudo-code de l'algorithme glouton

Complexité : $\mathbf{O(n^2)}$, soit $n-3$ itérations en $O(n)$ chacune.

D.2 : Caractère exact

La stratégie gloutonne **n'est pas exacte** : elle ne garantit pas la triangulation minimale.

Contre-exemple — l'heptagone de l'énoncé :

Méthode	Coût	Cordes choisies
Glouton	75,83	(s_3, s_5), (s_1, s_3), (s_0, s_3), (s_0, s_5)
Optimal	75,43	(s_0, s_2), (s_0, s_5), (s_2, s_5), (s_3, s_5)

Le glouton commence par $(s_3, s_5) \approx 15{,}65$, puis $(s_1, s_3) \approx 16{,}16$. Ces choix localement optimaux forcent ensuite $(s_0, s_3) \approx 21{,}93$ et $(s_0, s_5) \approx 22{,}09$, pour un total de 75,83. La solution optimale, en choisissant $(s_0, s_2) \approx 17{,}89$ et $(s_2, s_5) \approx 19{,}80$ à la place, obtient 75,43.

Résultats expérimentaux (polygone régulier) :

n	Coût DP	Coût Glouton	Ratio
10	968,21	984,05	1,016
20	1 586,24	1 626,43	1,025
50	2 415,15	2 463,41	1,020
100	3 043,06	3 094,65	1,017

Le glouton s'écarte d'environ **2 %** de l'optimal sur les polygones réguliers testés. Cet écart peut être plus important pour des polygones de géométrie défavorable.

E. Recommandation argumentée

Synthèse comparative

Critère	Essais successifs	Prog. dynamique	Glouton
Exactitude	Oui	Oui	Non ($\approx$2 %)
Temps	$O(2^{n(n-3)/2})$	$O(n^3)$	$O(n^2)$
Espace	$O(n)$ (pile)	$O(n^2)$	$O(n)$
Limite pratique	$n \leq 23$	$n \leq$ plusieurs 10^3	illimité
Reconstruction	Directe	Via table `choix`	Directe

Recommandation

La programmation dynamique est l'algorithme à recommander pour ce problème.

Ses atouts décisifs sont :

- **Exactitude garantie** dans tous les cas, quelle que soit la géométrie.
- **Complexité polynomiale** $O(n^3)$ garantie, non dépendante de l'élagage ni de la structure des données. Pour $n = 1\,000$, la DP termine en une fraction de seconde.
- **Mémoïsation efficace** : les $O(n^2)$ sous-problèmes sont chacun résolus une seule fois.
- La reconstruction des cordes optimales est $O(n)$ supplémentaire.

L'algorithme glouton, bien que très rapide ($O(n^2)$), ne donne aucune garantie d'optimalité. Pour une application où la précision est requise (robotique, maillage numérique, etc.), ce 2 % d'écart peut être critique.

Les essais successifs avec élagage restent précieux comme **outil de validation** de la DP sur de petits polygones, et pour explorer des variantes du problème où la programmation dynamique ne s'applique pas directement (polygones non convexes, distances non euclidiennes, contraintes additionnelles).

Rôle de chaque algorithme dans la pratique :

- **DP** $\rightarrow$ production (solution exacte, scalable)
- **Glouton** $\rightarrow$ initialisation rapide de la borne supérieure
- **Essais successifs** $\rightarrow$ validation et résolution de variantes contraintes

Conclusion

Nous avons étudié trois approches pour la triangulation minimale d'un polygone convexe.

La programmation dynamique est la solution recommandée : exacte, polynomiale $O(n^3)$, et adaptée à tout polygone de taille pratique. La clé de son efficacité est la décomposition en sous-polygones contigus partageant un sommet, qui garantit la validité, l'unicité et la mémoïsabilité des sous-problèmes.

Les essais successifs illustrent l'importance de l'élagage : sans la borne inférieure par longueur minimale (condition 4), la limite pratique est $n \approx 13$; avec elle, elle monte à $n \approx 23$. Le facteur de réduction pour $n = 13$ est de $\times 575$.

L'algorithme glouton est une heuristique rapide ($O(n^2)$) qui s'écarte d'environ 2 % de l'optimal sur les polygones réguliers. Il n'est pas exact : l'heptagone de l'énoncé en est un contre-exemple concret (75,83 contre 75,43).

Les 24 tests unitaires passent avec succès et valident la cohérence des trois approches : DP et backtracking donnent le même résultat optimal sur tous les polygones $n = 4$ à $n = 10$; le glouton est toujours $\geq$ DP ; les cordes restituées sont non-croisées et au nombre de $n - 3$.

Annexe — Architecture du code

Les fichiers sources sont disponibles dans le dépôt. Architecture :

```
src/
  polygon.rs      -- Structures de base (Point, distances, intersection)
  backtracking.rs -- B.1 valid_chord, B.3 backtrack avec elagage
  dynamic.rs      -- C.1/C.2 DP triangulation + reconstruction des cordes
  greedy.rs       -- D.1 algorithme glouton par selection d'oreilles
  main.rs         -- Programme principal : benchmarks et comparaisons
  tests.rs        -- 24 tests unitaires et d'integration
```

Compilation et exécution :

```
cargo build --release
cargo run --release    # Affiche les resultats pour toutes les sections
cargo test             # Lance les 24 tests
```